Frequent Itemsets 2

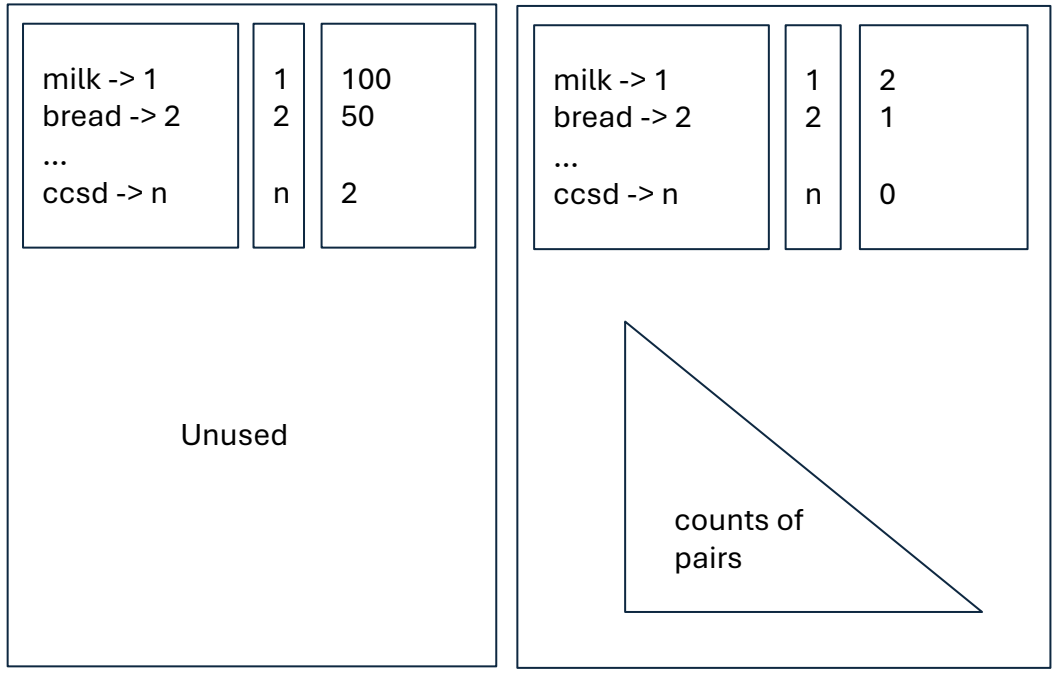
Introduction to Data Mining

Seoul National University

Spring 2026

Recap: Frequent Itemsets

- **Market-basket model:**
  - Association rule $I \rightarrow J$
  - Support, confidence & interest
- **A-Priori algorithm:**
  - Counting pairs is the hardest.
  - Key idea is *monotonicity*.
    - We can safely pass infrequent items.
  - Triangular vs triples method
  - Generalization to larger itemsets



<Pass 1> <Pass 2>

Outline

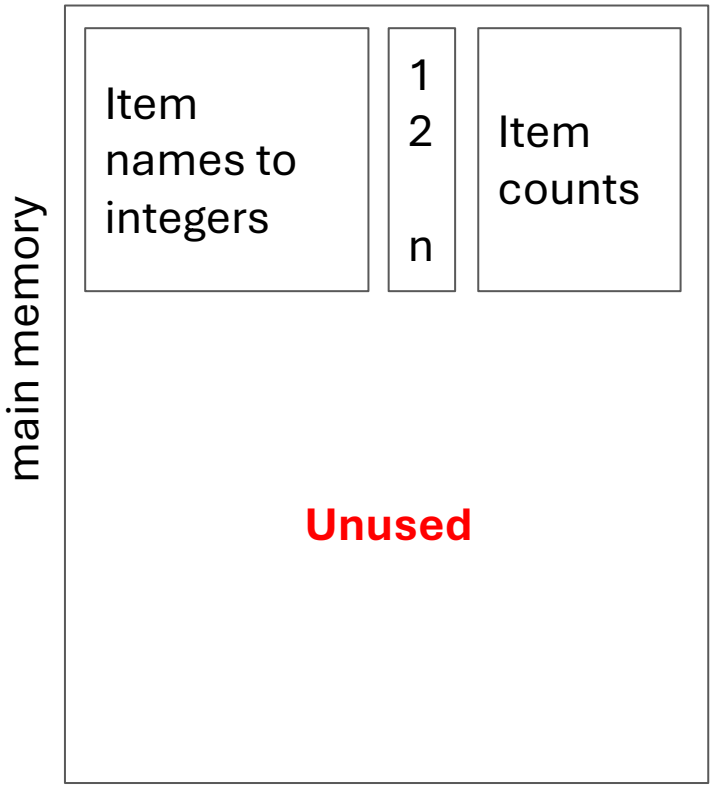
1. **<u>PCY Algorithm</u>**
2. Extensions of PCY
3. Limited-pass Algorithms

Handling Larger Data in Memory

- The A-Priori algorithm is fine as long as:
  - Counting candidate pairs C_2 can be done in main memory.
- **Q:** What if the memory is not enough?
- **A:** We need algorithms that cut down the size of C_2:
  1. PCY algorithm
  2. Multistage algorithm
  3. Multihash algorithm

PCY (Park-Chen-Yu) Algorithm

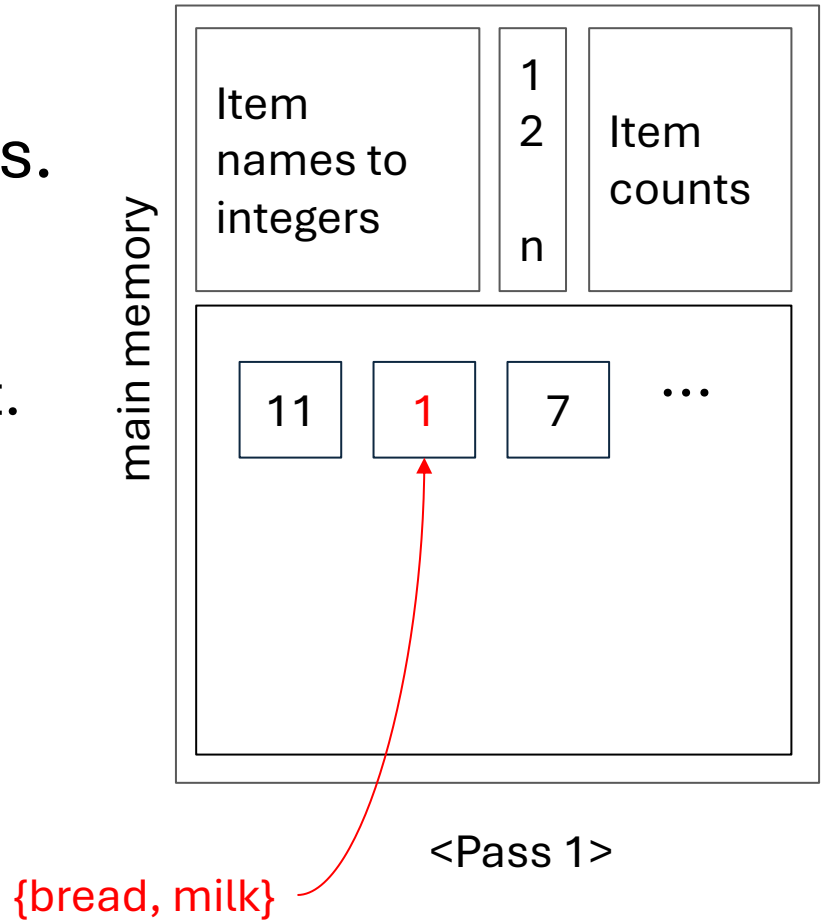
- In Pass 1 of A-Priori, most memory is idle.
  - We store only individual item counts.
- **Q:** Can we utilize the **idle memory**?
  - Probably for Pass 2.
  - Since we may not be able to store all pairs.



<Pass 1>

PCY (Park-Chen-Yu) Algorithm

- **PCY algorithm:** Create a **hash table** of pairs.
  - Make it have as many buckets as fit in memory.
    - E.g., 1,000 buckets for 4 KB.
  - Hash each pair and add 1 to the hashed bucket.
    - During Pass 1.
  - It's natural to have collisions.
    - # of buckets < # of unique pairs.



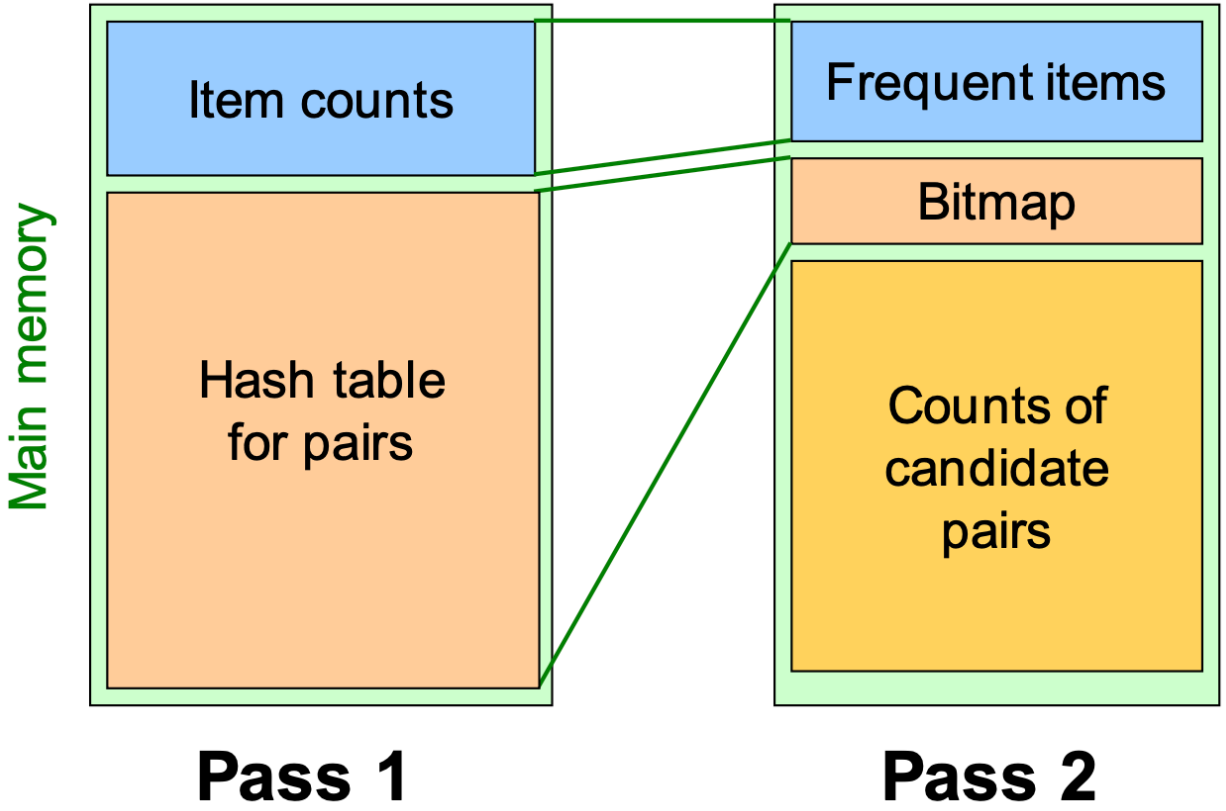
Observations about Buckets

- **Q:** Why do we do that?
- **A: To eliminate pairs that has no chance to be frequent.**
  - Each bucket B has a count of all pairs that hash to B.
    - If count $\geq$ support threshold s, pairs could all be frequent.
    - Otherwise, **no pair can be frequent!**
  - In Pass 2, define candidate pairs C_2 as $\{i, j\}$ such that:
    - Both i and j are frequent items.
    - $\{i, j\}$ hashes to a frequent bucket → **Difference from A-Priori.**
  - Later stages are similar to A-Priori.

Between Passes in PCY

- **Replace the buckets by a bit-vector:**
  - 1 means the bucket count exceeded the support threshold s.
  - 0 means it did not.
- **The bitmap reduces the space by 1/32.**
  - Since an integer is (normally) 32 bits.

Main Memory: Picture of PCY



Source: Stanford CS246 (2022)

One Subtlety

- PCY cannot use the triangular-matrix method for counting pairs.
  - Because PCY utilizes the *sparsity* of candidate pairs.
  - No way to “compact” a triangular matrix removing infrequent pairs.
- **So PCY is always forced to use the triples method.**

Pop Quiz

- Here is a collection of baskets: $\{1,2,3\}, \{3,4,5\}, \{2,4,5\}$.
- Suppose the support threshold is $s = 2$.
- On the first pass of PCY, we use a hash table with 3 buckets.
  - The pair $\{i, j\}$ is hashed to the bucket $[i \times j \bmod 3]$.
- **Q:** Which pairs are counted on the second pass of PCY?

Outline

1. PCY Algorithm
2. **<u>Extensions of PCY</u>**
3. Limited-pass Algorithms

Extensions of PCY

- **Two extensions of PCY:** Multistage and Multihash algorithms.
  - Both create multiple hash tables to reduce # of candidate pairs.
- The way they create multiple tables is different:
  - **Multistage:** Create tables through successive passes.
  - **Multihash:** Create tables at the same time (i.e., in parallel).

The Multistage Algorithm

- **Improves PCY by creating several successive hash tables:**
  - **Pass 1:** Same as PCY; results in a bitmap.
  - **Pass 2:** Create the second hash table to generate another bitmap.
    - This table has 31/32 of buckets due to the first bitmap.
    - Here a pair $\{i, j\}$ is hashed only if
      - Both i and j are frequent.
      - The pair is hashed to a frequent bucket in Bitmap 1.
  - **Idea:** The second table can contain much fewer frequent buckets.

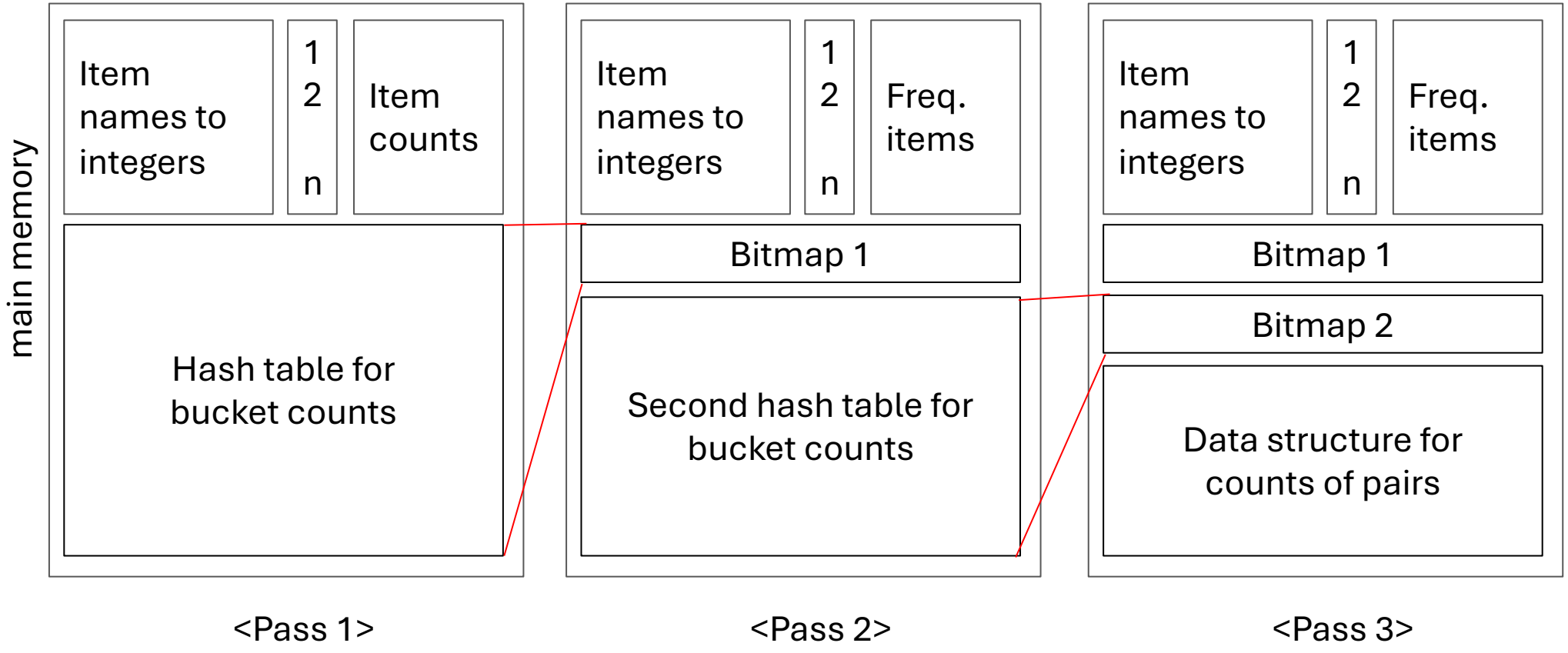
Conditions for the Candidates

- A pair $\{i, j\}$ is in C_2 if and only if
  1. Both i and j are frequent.
  2. Pair $\{i, j\}$ hashed to a frequent bucket in Bitmap 1.
  3. Pair $\{i, j\}$ hashed to a frequent bucket in Bitmap 2.

Conditions for the Candidates

- A pair $\{i, j\}$ is in C_2 if and only if
  1. Both i and j are frequent.
  2. Pair $\{i, j\}$ hashed to a frequent bucket in Bitmap 1.
  3. Pair $\{i, j\}$ hashed to a frequent bucket in Bitmap 2.
- **Q:** Is Condition 2 necessary? **Yes!**
  - Let's say $\{i, j\}$ is hashed to an infrequent bucket in Bitmap 1.
    - Then, it is not counted in Pass 2.
  - Doesn't mean $\{i, j\}$ cannot hash to a frequent bucket in Bitmap 2.
  - An infrequent pair $\{i, j\}$ may satisfy Conditions 1 and 3, but not 2.

The Multistage Algorithm



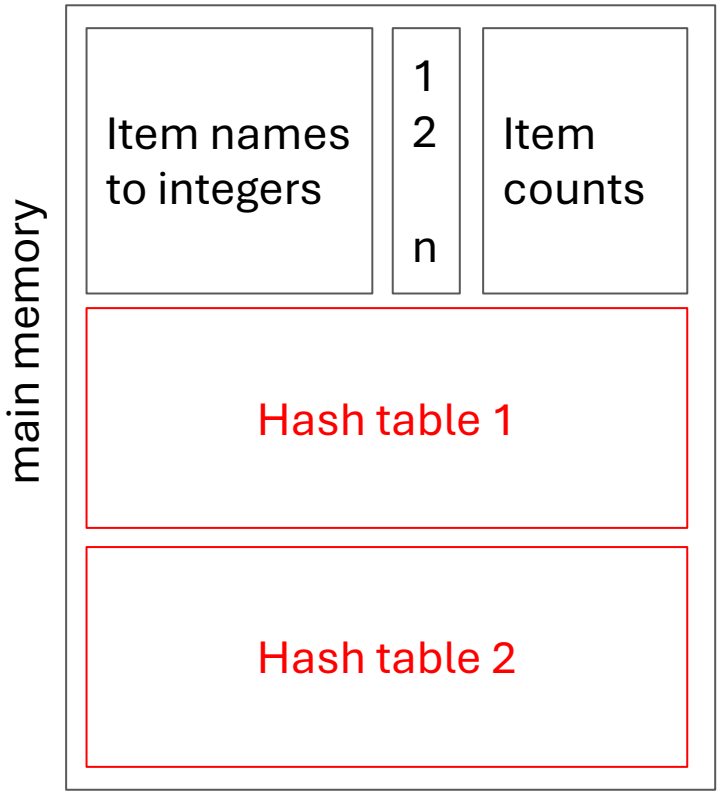
More Passes in Multistage

- We can add more passes until there is not enough space.
- Truly frequent pairs will always hash to frequent buckets.
  - No matter how many passes we use.

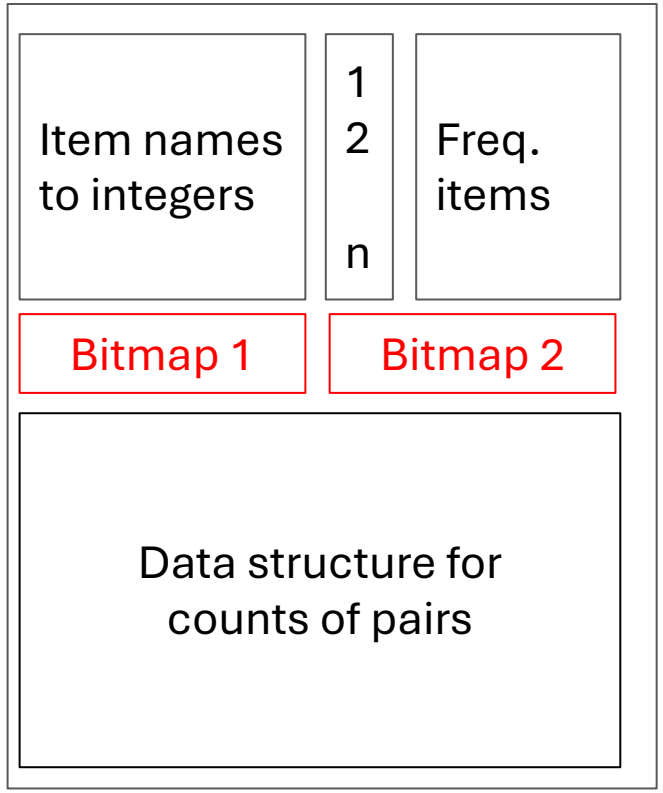
The Multihash Algorithm

- **Goal:** Get benefits of extra passes in a single pass.
- **Idea:** Create two hash tables with different hash functions.
  - Meaningful if the buckets are not too dense.
- A pair $\{i, j\}$ is in C_2 if
  1. Both i and j are frequent.
  2. The pair hashes to a frequent bucket in **both hash tables**.

The Multihash Algorithm



<Pass 1>



<Pass 2>

More Tables in Multihash

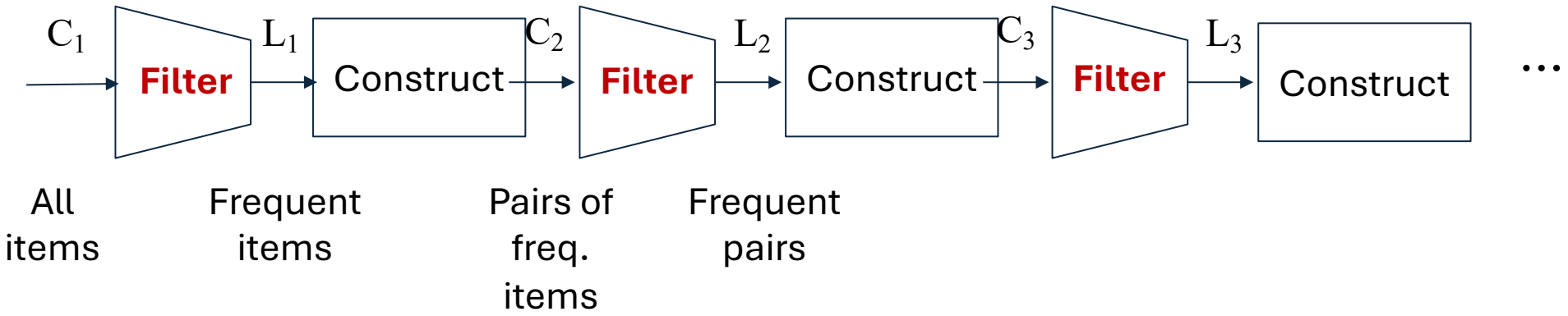
- We can increase the number n of tables as much as we want.
- **Q:** What is the optimal value of n?
- Let's say p^n is the probability of an infrequent pair being in C_2.
  - p: Probability of a pair to be frequent in each table.
- In limited memory, using more tables increases both n and p.
  - At some point, increasing # of tables increases the value of p^n.

Outline

1. PCY Algorithm
2. Extensions of PCY
3. **<u>Limited-pass Algorithms</u>**

Recap: Generalization of A-Priori

- So far, we used one pass for each size of itemsets.
  - The creation of L_k requires a new pass (i.e., Filter).
  - C_k: Set of candidate k-tuples that might be frequent sets.
  - L_k: Set of truly frequent k-tuples.



Limited-Pass Algorithms

- Many applications can sacrifice accuracy for speed.
  - E.g., it's enough to find most frequent itemsets in a supermarket.
- We can find all or most frequent itemsets using ≤ 2 passes.
  - **Random sampling:** Simplest approach.
  - **Toivonen:** Two passes on average, but also may not terminate.
  - **SON:** Divide the data into multiple chunks.

Random Sampling

- **Idea:** Take a random sample of the market baskets.
- Run A-Priori or one of its improvements in main memory:
  - If the sample ratio is $p < 1$, adjust support threshold s to ps.

B1 = {m, c, b}
B2 = {m, p, j}
B3 = {m, c, b, n}
B4 = {c, j}
B5 = {m, p, b}
B6 = {m, c, b, j}

Sample by p

B1 = {m, c, b}
B3 = {m, c, b, n}
B4 = {c, j}

A-Priori (or any other algorithm) using support threshold ps.

Avoiding Errors in Sampling

- Sampling introduces both *false positives* and *false negatives*.
  - **False positives:** Itemsets frequent in sample, but not in the whole.
  - **False negatives:** Itemsets frequent in the whole, but not in sample.
- **Eliminate false positives** by making another full pass.
  - Count all frequent itemsets in sample.
- **Reduce false negatives** by reducing threshold (say to $0.9ps$).
  - No easy way to eliminate false negatives.

Toivonen's Algorithm

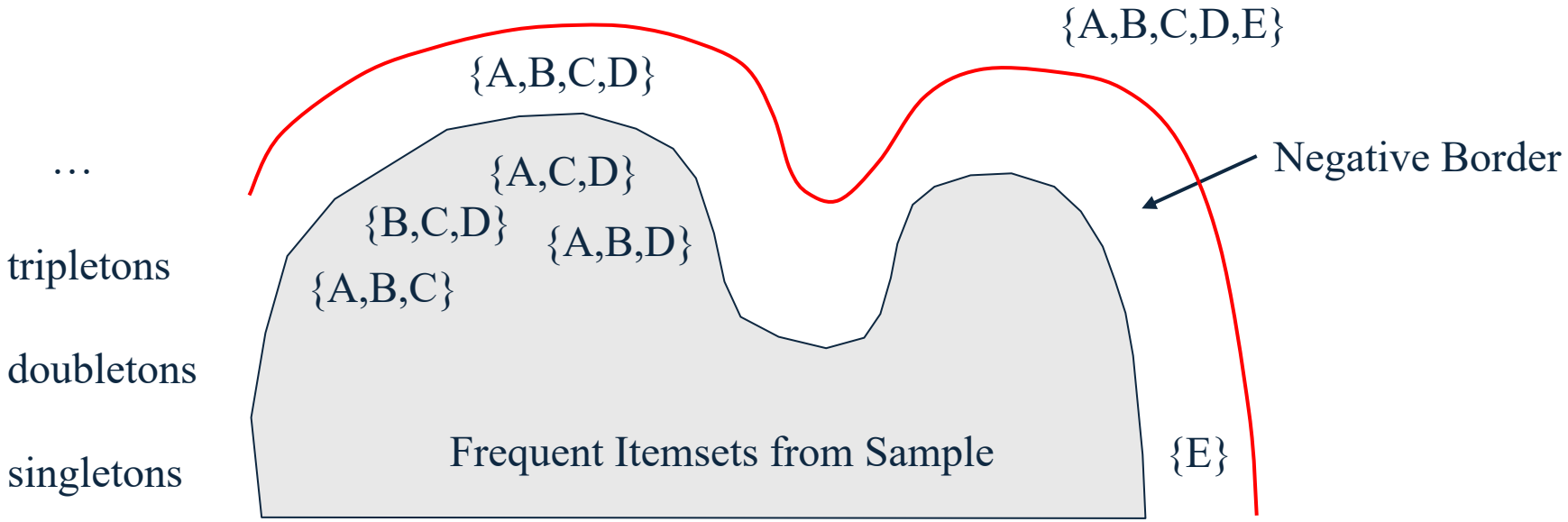
- **Q:** Can we eliminate false negatives at all?
- **Idea:** Construct *negative border* for an efficient safety check.
- **Toivonen's algorithm:**
  - Use one pass over samples and one pass over the full data.
  - No false negatives or positives.
  - However, there is a small chance it will not finish by 2 passes.
    - The average number of passes to produce answer is still a constant.

Toivonen's Algorithm (Pass 1)

- Find candidates from a small sample.
  - Use support threshold less than proportional value ps (say $0.9ps$).
- Construct the **negative border**.
  - Collection of itemsets that are not frequent in sample, but all of their immediate subsets are frequent in sample.
  - **Immediate subsets:** Constructed by deleting exactly one item.

Negative Border Example

- $\{A, B, C, D\}$ is in the negative border if and only if:
  - It is not frequent in the sample, but
  - All of $\{A, B, C\}$, $\{B, C, D\}$, $\{A, C, D\}$, and $\{A, B, D\}$ are.
- $\{E\}$ is in the negative border if it is not frequent.
  - Its immediate subset $\emptyset$ is considered frequent.



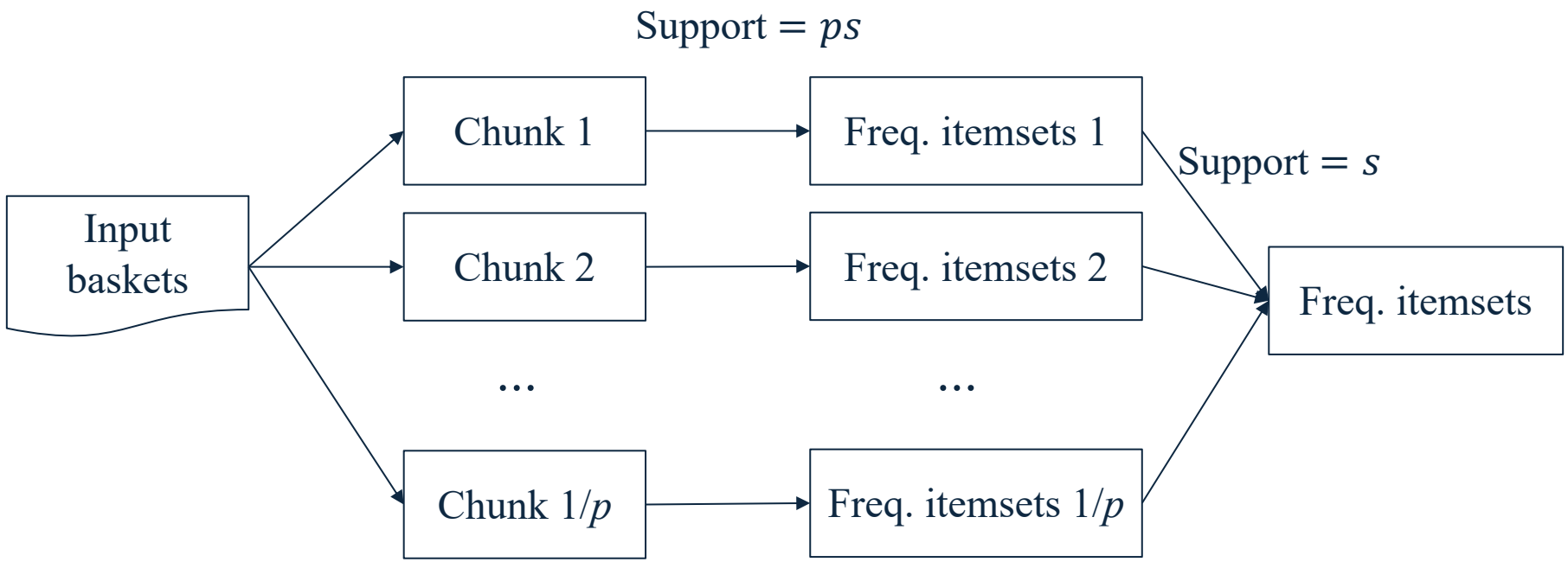
Toivonen's Algorithm (Pass 2)

- Make a pass on the entire dataset.
  - Count itemsets (a) frequent in sample or (b) existing in negative border.
- **Finish** if no member of negative border is frequent in full data:
  - We have found the correct set of frequent itemsets
- **Otherwise,** there may be a larger set that is frequent.
  - Do not finish, and repeat the algorithm with new random sample.

Why Toivonen's Algorithm Works

- **No false positives:** Clearly because of Pass 2.
- **No false negatives:**
  - Suppose an itemset S frequent in whole data, but not in sample.
  - Then, the border contains at least one itemset frequent in the whole.
  - **Why?** One of the following holds:
    - S is in the negative order.
    - S has an immediate subset T frequent in whole data, but not in the sample.
      - This goes recursively until there is a subset in the negative order.
      - Recall that every singleton is in the negative order.

SON (Savasere, Omiecinski, Navathe)



SON Algorithm

- Avoids both false negatives and positives with two full passes.
- **Pass 1:**
  - Divide input file into $1/p$ chunks.
  - Run A-Priori with ps as support threshold.
- **Pass 2:**
  - Take union of all frequent itemsets and select those with support $\geq s$.
- **There are no false negatives!**
  - A frequent itemset must be frequent in at least one chunk.
  - If an itemset is not frequent in any chunk, its support is $< (1/p)ps = s$.

SON with MapReduce

- MapReduce (or any other parallel computation) can be used.
- **Pass 1:** Find candidate itemsets.
  - **Map:** Take a chunk and return $(F, 1)$ for each frequent itemset F.
    - Support threshold is ps.
  - **Reduce:** Ignore value and produce itemsets that appear at least once.
- **Pass 2:** Find true frequent itemsets.
  - **Map:**
    - Take candidate itemsets from Pass 1 and a portion of the input data.
    - Return (C, v) where C is a candidate itemset and v is the support for this portion.
  - **Reduce:** For each itemset, sum the values and output if the sum $\geq s$.

Summary

1. PCY Algorithm
2. Extensions of PCY
   - Multistage algorithm
   - Multihash algorithm
3. Limited-pass Algorithms
   - Random sampling
   - Toivonen's algorithm
   - SON algorithm